

Repository Summary, Code Sample

Yufeng (Alex) Xia · yufeng.xia@ed.ac.uk · University of Edinburgh

Two repositories, chosen to show two different things: what I build alone and own end to end, and how I work inside a large shared codebase under review.

FLUX-DT, BUILT ON OCUDU (SOLE AUTHOR)

- **URL** github.com/xyf2002/OCUDU/tree/evaluation
- **What it is** My MSc by Research thesis system, built by modifying OCUDU, the Linux Foundation's open-source 5G CU/DU stack (C++17, 3GPP and O-RAN compliant). I added a GPU ray-tracing digital twin *into the radio path of the live stack*, together with the runtime that lets it survive running on compute it does not own.
- **The problem** Spare compute in a RAN is real but reclaimable at any moment, and a radio twin is stateful, so killing it loses scene state and progress. The runtime therefore has to degrade rather than die: admit work selectively, drop fidelity in graded steps, checkpoint, and hand capacity back on the timescale the live radio imposes.
- **Stack** C++17 (OCUDU CU/DU stack, CMake) with a **Python 3.10** control plane packaged as `ocudu-gpu-backend`; **NVIDIA Sionna RT** for GPU ray tracing; NumPy for the channel maths (CIR collapse, zero-forcing, SINR); a hand-written TCP transport (length-framed messages, TCP_NODELAY, binary payload encoding) between control plane and GPU workers; `concurrent.futures` and threading for the worker pool; `mypy --strict` and `pytest` across the package.
- **Scale and ownership** Roughly the 500 most recent commits on this branch are mine, sole-authored.

What I built

- **Admission and selection** (`gpu_backend/admission/`): A joint admission objective across co-channel cells, solved with a greedy selection heuristic against per-tier fidelity thresholds, with an admission cache for repeated decisions and baseline policies to compare against. The same ranking that decides which tiles are worth tracing under interference also gives the order in which work is shed when compute is reclaimed mid-round.
- **Solver orchestration** (`gpu_backend/solve/`): Round-based partitioned solving with an explicit runtime plan, so a round can be cut short without losing what it already produced. Hedged dispatch lets a replica that lands early end the round instead of waiting on a straggler.
- **Worker pool and health** (`gpu_backend/workers/`): Dispatch balancing across GPU workers, per-worker batch timing so a worker is charged for its own work rather than the round it sat in, and health and timeout handling that reclaims GPU slots from a stalled worker instead of leaking them.
- **Checkpointing and recovery** (`gpu_backend/checkpoint/`): Checkpoint store, on-disk layout and validation, plus bounded recovery when the backend disappears underneath a running twin. This is what makes preemption survivable rather than fatal.
- **Scene caching and pruning** (`gpu_backend/scene_processing/`, `cache/`): Scene-identity hashing, mesh trimming and geometry subsetting, so an unchanged scene is recognised before it is re-hashed and tiles that are never traced are never priced.
- **Channel pipeline and integration** (`gpu_backend/channel/`, `servers/`): Ray-traced bundles turned into interference and SINR (zero-forcing, constellation, frame processing) and applied at the RF frontend of the live stack, behind control, worker, cache and admin services.
- **Evaluation harness** (`evaluation/`): Replays measured RAN GPU occupancy as a contending load beside the twin, and sweeps elasticity, fidelity, multi-cell contention and recovery, with the plotting that turns those runs into thesis figures.

- **URL** github.com/netsys-edinburgh/morphling
- **What it is** An emulator for distributed machine-learning training on heterogeneous edge devices. It runs *unmodified* training scripts against a unified real-or-emulated backend, fits performance models from real measurements, and drives execution on event-driven virtual time, so large-scale what-if studies need no physical device fleet.
- **Status** Research artifact of *Morphling: Emulator for Distributed Machine Learning at the Edge*, EdgeSys'26 (co-located with ACM MobiSys 2026), **Best Paper Award**. DOI 10.1145/3812836.3814779. ~18,000 lines of C++/CUDA plus a Python surface, CMake, CI with CodeQL and numerical-consistency checks.
- **Stack** C++17 and CUDA built with CMake against LibTorch and the CUDA Toolkit, exposed to Python through a native extension; libevent for the event-driven transport; Protocol Buffers (vendored, with FlatBuffers evaluated against it) for the wire format; pinned host memory via cudaHostAlloc; CI covering build, formatting, CodeQL, protobuf breaking-change and numerical-consistency checks.
- **My share** 91 of 519 commits, merged through reviewed pull requests, over one clearly delimited subsystem: **the inter-device transport data plane and its performance**.

What I owned

- **Event-driven transport** (`csrc/backend/proxy_svr.cc`, `proxy_cli.cc`, `server_base.cpp`): The proxy server and client that carry tensor traffic between emulated devices, built on libevent: non-blocking connection handling, event-loop threads, batched send and receive accounting, and connection lifecycle management. Emulation fidelity is bounded by this path, because a transfer that costs more than the emulated device would have spent makes the emulated timeline wrong, so the work was about driving transfer cost down and making it predictable.
- **Zero-copy and pinned-memory path**: A CudaPinnedMemoryPool allocating cudaHostAlloc buffers in power-of-two buckets and recycling them, which takes allocation out of the hot path; zero-copy scatter-gather buffers (`evbuffer_add_reference` with `shared_ptr` cleanup) so payloads move without staging copies; CUDA host registration and worker-thread pinning, each added only after a dedicated microbenchmark showed it paid for itself.
- **Serialisation cost study**: Instrumented the full send path (serialise, compute size, transmit) and compared Protocol Buffers against FlatBuffers at realistic message sizes, then picked the framing and batching strategy from those measurements. FlatBuffers turned out to be *slower* in the Python path, against the usual assumption, and the result is documented in the repository.
- **Distributed virtual-clock synchronisation** (`csrc/base/virtual_clock.cc`): Virtual time has to stay consistent across emulated devices or the emulator silently produces wrong results. I implemented the synchronisation mechanism and compared alternative strategies against each other.
- **Failure modes that only appear at scale**: Diagnosed and fixed deadlocks arising under device failure and connection teardown, and the device-scale-zero condition, the class of bug that never reproduces on two nodes.
- **Measurement tooling**: Per-device timeline and Gantt visualisation of emulated execution, used both for debugging and for figures in the paper.